AddressSanitizer 第一阶段复现说明

本机环境、错误检测验证、开源替代基准与单因素扩展

2026-10-25 | 阶段报告，不等同于最终提交

周潇洲 1240025753 | 张瀚元 1240025908
章殷远 1230022374 | 章越翔 1230019110

本阶段结论

总体判定：本阶段部分复现了 ASan 的检测行为和开销测量方法，原论文的 SPEC 数值尚不能判定为已复现。工具链和可追溯的第一轮实验已完成；错误案例正确报错，合法访问和修复控制均正常退出。性能实验使用开源 Olden 子集，不能替代原论文的 SPEC CPU2006 平均值。

表 1. 阶段证据摘要

项目	已完成的证据
错误与合法控制	45 / 45 passed; 5 runs per case
性能实验	5 workloads x 3 configurations x 5 runs = 75 trials
扩展实验	2 workloads x 3 redzone settings x 5 runs = 30 trials
时间倍率均值	Modern: 7.663x; paper-like: 2.243x

这里的时间倍率为各工作负载配对倍率均值的等权算术平均，不是 SPEC 得分。所有图表都从本地日志生成；没有填入估计结果。

证据目录 / Evidence: records\stage1\20261025T072502Z-06ebb2

1. 环境与复现范围

为什么沿用 WSL？原论文在 64 位 Linux 上评估，本机已有可用的 Ubuntu。复用它能减少平台差异；WSL 的调度和虚拟化影响仍要记录。旧 PERIOD 文件完整归档，两个旧镜像已删除，Ubuntu 虚拟磁盘未删除或压缩。

表 2. 本机实验环境

项目 / Item	本次设置 / This study
Host	Windows 11, Intel Core i5-13500HX (14 cores / 20 threads), about 15.7 GiB usable RAM
Guest	Ubuntu 22.04.5 LTS; WSL2 kernel 6.18.54.1; 6 visible CPUs; 6 GiB memory limit
Tools	Clang/LLVM 14.0.0 (Ubuntu 1:14.0.0-1ubuntu1.1); CMake 3.22.1; Ninja 1.10.1; lit 14.0.6
Benchmark commit	d52a93a891cde7a76aca63f7a201c5a5fe459f41
Workloads	bisort, mst, perimeter, treeadd, tsp; unmodified upstream Olden sources; reference input size
Execution	Linux ext4 work directory; one Linux logical CPU pinned; sequential measured processes; randomized block order, seed 340

与原论文的差异 / Deviations from the paper

表 3. 与原论文设置的差异

项目 / Item	原论文 / Paper	本次 / Ours; reason
Hardware/OS	Dual Xeon E5620, 24 GB, Linux	Laptop / WSL2; available local machine
Tool/workload	2012 implementation; SPEC CPU2006	Clang 14; Olden; no institutional SPEC license; instructor allowed substitution
Optimization	clang -O2	-O2, frame pointers, non-PIE for both variants; fixed executable layout and readable traces
Memory metric	VmPeak at termination	Peak RSS from GNU time; measures resident memory and is not directly comparable
Coverage	Full SPEC suite, writes-only mode	Five Olden programs; full read/write ASan only; bounded first stage

两个版本共享优化和布局设置，ASan 版本额外开启地址插桩。性能版本关闭 UAR 和 use-after-scope 扩展，避免把 2012 年论文之外的功能混入主范围。

构建兼容性：直接配置 Olden 子目录时，需在生成的构建树补充 lit.local.cfg，设 traditional_output=True；未改动上游 C 源码。性能输出先经管道捕获，计时文件先写 Linux ext4，再保存到仓库，以减少 Windows 文件系统对计时的影响。

2. ASan 如何工作及错误检测结果

Clang 在内存访问附近插入检查。影子内存记录对应字节是否允许访问；对象周围的 redzone 被标记为不可访问。释放后的内存可暂留在 quarantine 中，帮助发现释放后使用。运行库在发现错误时输出类型、地址和调用栈，然后退出。

这是“检测测试实际触发的内存错误”，不是静态证明程序没有错误。每个错误独立运行，因为 ASan 通常在第一个错误后退出。

表 4. 错误检测和合法控制

测试 / Case	期望 / Expected	通过 / Passed
heap	heap-buffer-overflow	5/5
stack	stack-buffer-overflow	5/5
global	global-buffer-overflow	5/5
uaf	heap-use-after-free	5/5
double_free	attempting double-free	5/5
control-heap	clean exit	5/5
control-stack	clean exit	5/5
control-global	clean exit	5/5
control-fixed	clean exit	5/5

判定不仅要求非零退出，还要求错误类别匹配，并且栈信息含源文件和行号。合法的数组下标为 7；越界下标为 8。fixed 控制把输入约束到合法下标，验证修复后正常退出。

本阶段只支持这五类构造案例的检测能力。有限的合法控制没有误报，不等于证明 ASan 对所有程序均无误报。原论文发现 Chromium 中数百个未知缺陷的结果没有复现。

Traceability: correctness.json -> correctness/<case>-<run>.log; examples/memory_cases.c; source-snapshot/.

3. 性能实验与结果

每个程序先预热一次，再对原生版、现代默认风格 ASan 和接近论文参数的 ASan 各运行 5 次。每轮以固定随机种子打乱顺序，降低固定运行顺序带来的偏差。所有测量输出都要求一致，且通过上游参考输出校验。

时间包括启动与退出，使用单调高精度计时器；过短的程序可能受启动成本影响。倍率按同一轮 ASan 时间除以原生时间，再计算均值和样本标准差，不能把标准差当作误差上界。

表 5 与图 1 展示配对时间倍率；完整的各配置秒数在原始记录中。

表 5. 配对时间倍率及样本标准差

程序 / Workload	原生秒 / Native s	Modern ASan / native	Paper-like / native
bisort	0.253 +/- 0.050	2.447 +/- 0.496x	1.770 +/- 0.267x
mst	0.030 +/- 0.005	1.884 +/- 0.251x	1.841 +/- 0.077x
perimeter	0.092 +/- 0.016	6.000 +/- 0.783x	1.913 +/- 0.111x
treeadd	0.106 +/- 0.019	24.728 +/- 5.356x	3.830 +/- 0.807x
tsp	0.266 +/- 0.027	3.260 +/- 0.288x	1.862 +/- 0.134x

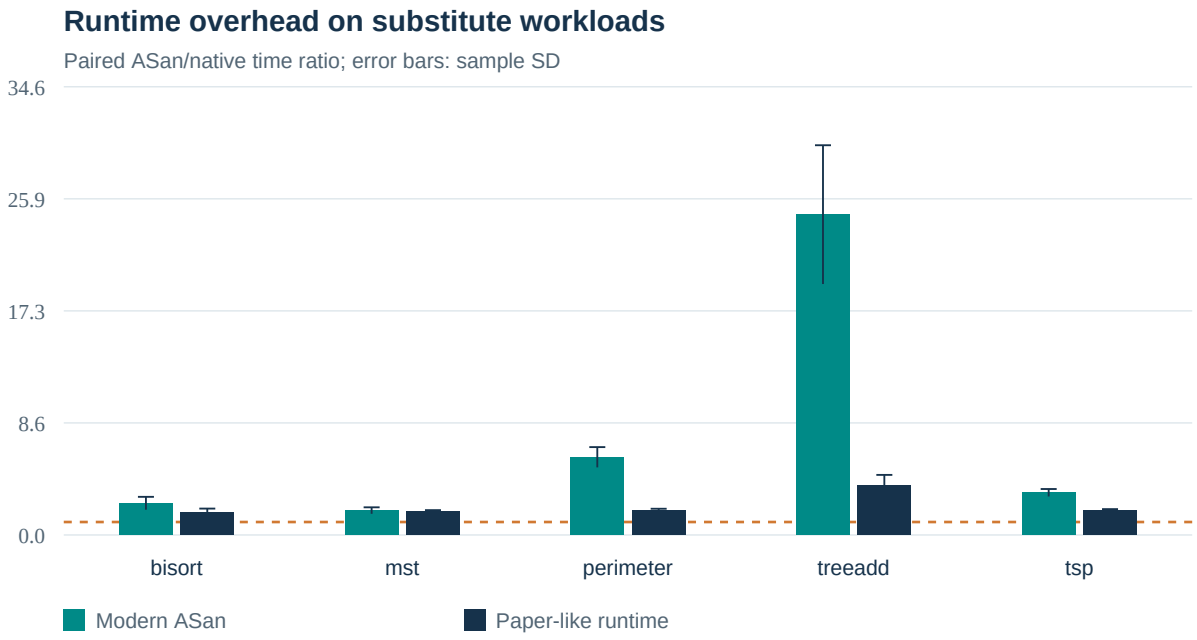

Figure 1. Paired time ratios; bars show means and whiskers show sample SD. Native reference = 1.0.

现代配置保留运行库的分配追踪、redzone 和 quarantine 默认行为，显式关闭内存泄漏检测。接近论文的配置设 redzone=max_redzone=32、quarantine=0、malloc_context_size=0。两者都不是 2012 年二进制。

Traceability: measurements.csv / .json -> performance/<workload>/<configuration>-<run>.log and .time. Warm-ups are recorded separately.

4. 内存、二进制大小及参数解释

峰值 RSS 是进程实际驻留内存的高水位，和原论文表 1 使用的虚拟地址空间峰值 VmPeak 不同。现代 64 位 ASan 映射很大的影子地址空间，不能把映射的虚拟空间直接当作消耗的物理内存。

表 6. 峰值驻留内存和文件大小

程序 / Workload	Native RSS MiB	Modern RSS MiB	Paper-like RSS MiB	No-debug file ratio
bisort	17.83 +/- 0.03	131.61 +/- 0.01	40.46 +/- 0.05	85.07x
mst	26.36 +/- 0.00	36.48 +/- 0.02	36.24 +/- 0.01	85.45x
perimeter	86.86 +/- 0.00	282.32 +/- 0.05	124.46 +/- 0.05	86.57x
treeadd	129.48 +/- 0.00	1088.57 +/- 0.04	292.56 +/- 0.06	86.96x
tsp	66.06 +/- 0.06	316.81 +/- 0.04	112.78 +/- 0.03	68.74x

二进制倍率按移除调试信息后的完整文件大小计算，每个版本只构建一次，因此没有重复测量的标准差；它包含 ASan 运行库等固定成本，小程序的倍率可能很大。还保存了 llvm-size 的节区数据。

为什么不能把所有检查都关掉再测？

缩小 redzone 会减少保护范围；关闭 quarantine 可能降低释放后使用的可检测性；关闭分配栈记录会减少诊断信息。论文的性能设置就是一种取舍，不能把它当成最强检测配置。错误检测案例使用保留诊断信息的现代配置。

本次基准输入 / Inputs used

表 7. 上游参考输入参数

Workload	Arguments from generated upstream .test
bisort	700000
mst	1000
perimeter	10
treeadd	22
tsp	1024000

Traceability: binary-sizes.json; sizes/*-sections.log; workloads.json; build/*-reference-validation.log; environment/runtime-files.log.

5. 单因素扩展：最小堆 redzone

问题：在相同程序和输入下，把最小堆 redzone 从 16 改为 32、128 字节，会怎样影响时间和内存？只改变 redzone 一个参数；max_redzone 固定 2048，quarantine 和分配栈记录均固定为 0。这里测试的是最小值，因为现代分配器会自适应选择 redzone。

结果见表 8 与图 2；主实验的两套运行配置同时改变多个设置，不能用来单独归因。

表 8. 最小堆 redzone 单因素结果

Workload	Minimum redzone	Time, s (mean +/- SD)	Peak RSS, MiB (mean +/- SD)
bisort	16	0.420 +/- 0.076	31.46 +/- 0.03
bisort	32	0.428 +/- 0.021	40.46 +/- 0.03
bisort	128	0.607 +/- 0.127	94.49 +/- 0.01
mst	16	0.054 +/- 0.010	36.47 +/- 0.03
mst	32	0.050 +/- 0.001	36.48 +/- 0.01
mst	128	0.057 +/- 0.014	36.46 +/- 0.05

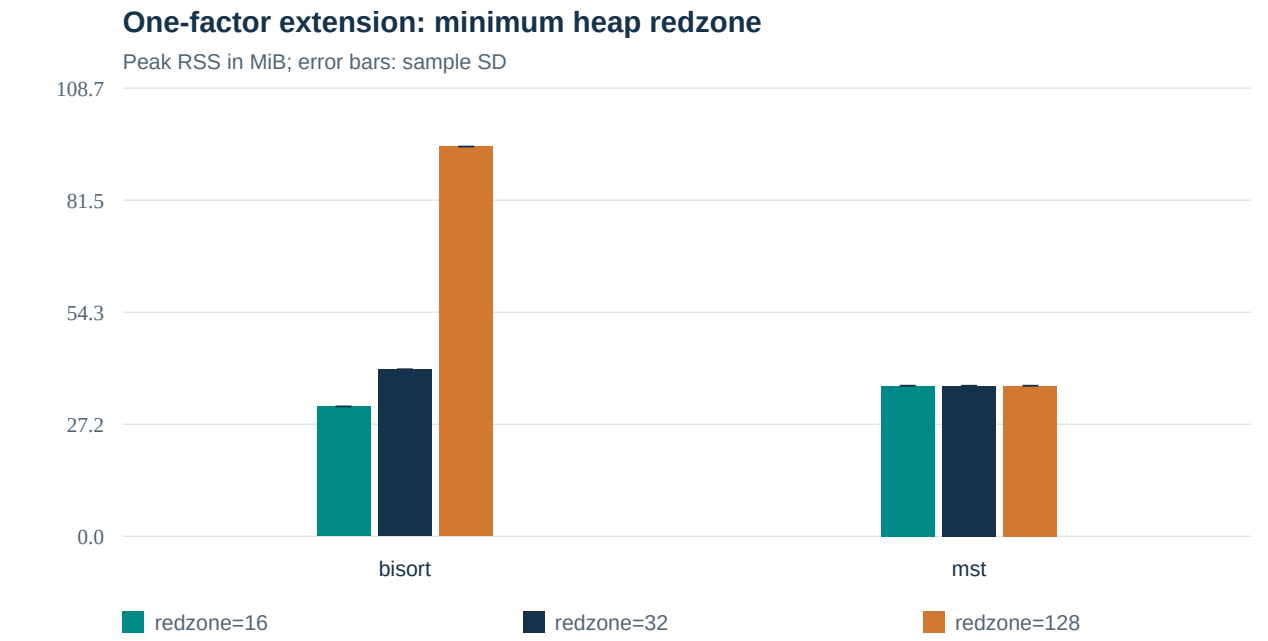

Figure 2. Measured peak RSS under one changed runtime parameter; means and sample SD, n=5.

16 到 128 字节时的平均 RSS 变化：bisort: 3.00x (+200.4%); mst: 1.00x (-0.0%).

这能检验“ redzone 设置会影响资源开销 ”的局部结论；它没有估计真实漏洞检出率，也不能证明所有负载都呈相同趋势。

6. 与论文对照、限制与后续工作

表 9. 论文主张与本阶段证据对照

主张 / Claim	论文 / Paper	本次 / Ours	判定 / Verdict
Fault categories (Sec. 3)	Heap/stack/global bounds; UAF	5 constructed faults detected, valid controls clean	Supported only for tested cases
Runtime (Fig. 2)	1.73x average on SPEC CPU2006	7.663x modern; 2.243x paper-like on Olden	Original numeric claim inconclusive: different workloads, compiler, host
Memory (Table 1)	3.37x aggregate VmPeak	Peak RSS measured separately	Inconclusive: different metric and workload
Tuning (Sec. 4.3)	Redzones affect resources	Controlled redzone experiment above	Evidence restricted to two inputs

仅作描述：本次接近论文参数的等权时间倍率与 1.73 的数值差为 +0.513x（相对于 1.73 为 +29.7%）。因工作负载不同，这不是同设置下的复现误差，不能把差值归因于某一种硬件或版本。

受控检查：同一编译器、源代码、输入、优化和 CPU 亲和性；上游参考输出验证；现代与论文参数配置对照；redzone 单因素重跑。仍未控制 Windows 后台任务、温度与功耗、WSL 虚拟 CPU 到大小核的映射。五个程序均来自 Olden，代表性有限。

后续与课程待办 / Next steps and course requirements

下一阶段应扩展到不同类型的 C/C++ 应用，增加输入规模，并在低负载时复测短程序。SPEC 原始数值、只插桩写操作、Chromium 未知缺陷发现仍不在本阶段结论内。最终报告需保持英文、至少三次运行统计、偏差表及可追溯日志 [4]。

小组仓库为 github.com/zxz-codes/CS340-AddressSanitizer-Replication，四位成员信息已登记。按小组要求，个人分工暂不填写。每位成员仍需从自己的账号进行实质提交，提交信息包含学号和姓名；每周日 23:59 检查，最终才打 v1.0。本报告是阶段成果，不代表最终提交已完成。

贡献与 AI 使用 / Contributions and AI use

小组提供选题、课程材料和教师对基准替代的回复。AI 协助归档、安装、编写脚本与案例、执行实验、分析日志和制作本报告；数字来自实际运行。四位成员的设计、复核、分析与展示贡献待本人核对填写。

References

[1] Serebryany et al. AddressSanitizer: A Fast Address Sanity Checker. USENIX ATC 2012, Secs. 3-4, Fig. 2, Tables 1-2. https://www.usenix.org/system/files/conference/atc12/atc12-final39.pdf

[2] Clang AddressSanitizer documentation. https://clang.llvm.org/docs/AddressSanitizer.html (accessed 25 Oct 2026).

[3] LLVM test-suite guide and repository, pinned commit above. https://llvm.org/docs/TestSuiteGuide.html ; https://github.com/llvm/llvm-test-suite

[4] CS340 Research Paper Replication, Academic Year 2026-2027, supplied course handout; instructor reply supplied as screenshot permits an open-source benchmark substitute.